

第 8 章

形态学操作

形态学，即数学形态学（Mathematical Morphology），是图像处理过程中一个非常重要的研究方向。形态学主要从图像内提取分量信息，该分量信息通常对于表达和描绘图像的形状具有重要意义，通常是图像理解时所使用的最本质的形状特征。例如，在识别手写数字时，能够通过形态学运算得到其骨架信息，在具体识别时，仅针对其骨架进行运算即可。形态学处理在视觉检测、文字识别、医学图像处理、图像压缩编码等领域都有非常重要的应用。

形态学操作主要包含：腐蚀、膨胀、开运算、闭运算、形态学梯度（Morphological Gradient）运算、顶帽运算（礼帽运算）、黑帽运算等操作。腐蚀操作和膨胀操作是形态学运算的基础，将腐蚀和膨胀操作进行结合，就可以实现开运算、闭运算、形态学梯度运算、顶帽运算、黑帽运算、击中击不中等不同形式的运算。

8.1 腐蚀

腐蚀是最基本的形态学操作之一，它能够将图像的边界点消除，使图像沿着边界向内收缩，也可以将小于指定结构体元素的部分去除。

腐蚀用来“收缩”或者“细化”二值图像中的前景，借此实现去除噪声、元素分割等功能。例如，在图 8-1 中，左图是原始图像，右图是对其腐蚀的处理结果。

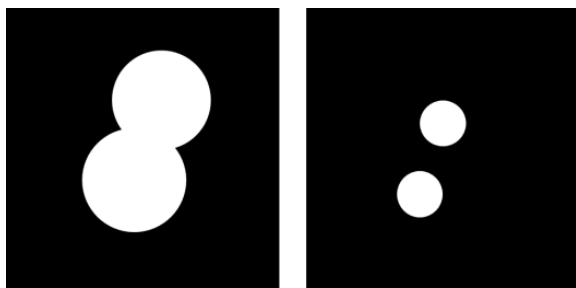


图 8-1 腐蚀示例

在腐蚀过程中，通常使用一个结构元来逐个像素地扫描要被腐蚀的图像，并根据结构元和被腐蚀图像的关系来确定腐蚀结果。

例如，在图 8-2 中，整幅图像的背景色是黑色的，前景对象是一个白色的圆形。图像左上角的深色小方块是遍历图像所使用的结构元。在腐蚀过程中，要将该结构元逐个像素地遍历整幅图像，并根据结构元与被腐蚀图像的关系，来确定腐蚀结果图像中对应结构元中心点位置的像素点的值。



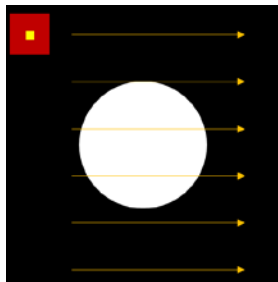


图 8-2 结构元与被腐蚀图像

需要注意的是，腐蚀操作等形态学操作是逐个像素地来决定值的，每次判定的点都是与结构元中心点所对应的点。

图 8-3 中的两幅图像表示结构元与前景色的两种不同关系。根据这两种不同的关系来决定，腐蚀结果图像中的结构元中心点所对应位置像素点的像素值。

- 如果结构元完全处于前景图像中（图 8-3 的左图），就将结构元中心点所对应的腐蚀结果图像中的像素点处理为前景色（白色，像素点的像素值为 1）。
- 如果结构元未完全处于前景图像中（可能部分在，也可能完全不在，图 8-3 的右图），就将结构元中心点对应的腐蚀结果图像中的像素点处理为背景色（黑色，像素点的像素值为 0）。

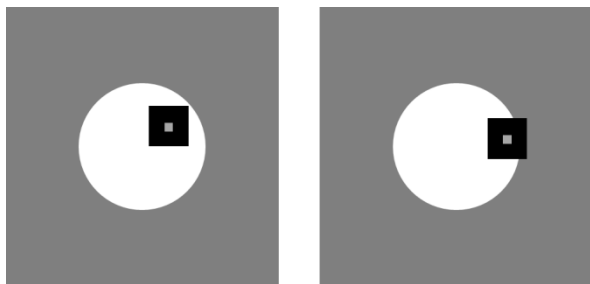


图 8-3 腐蚀原理

针对图 8-3 中的图像，腐蚀的结果就是前景色的白色圆直径变小。上述结构元也被称为核。

例如，有需要被腐蚀的图像 `img`，其值如下，其中 1 表示白色前景，0 表示黑色背景：

```
[[0 0 0 0 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 0 0 0 0]]
```

有一个结构元 `kernel`，其值为：

```
[[1]
 [1]
 [1]]
```

如果使用结构元 `kernel` 对图像 `img` 进行腐蚀，则可以得到腐蚀结果图像 `rst`：

```
[[0 0 0 0 0]
```



```
[0 0 0 0 0]
[0 1 1 1 0]
[0 0 0 0 0]
[0 0 0 0 0]]
```

这是因为，当结构元 `kernel` 在图像 `img` 内逐个像素遍历时，只有当核 `kernel` 的中心点“`kernel[1,0]`”位于 `img` 中的 `img[2,1]`、`img[2,2]`、`img[2,3]`时，核才完全处于前景图像中。所以在腐蚀结果图像 `rst` 中，只有这三个点的值被处理为 1，其余像素点的值被处理为 0。

上述示例如图 8-4 所示，其中：

- 图(a)表示要被腐蚀的 `img`。
- 图(b)是核 `kernel`。
- 图(c)中的阴影部分是 `kernel` 在遍历 `img` 时，`kernel` 完全位于前景对象内部时的 3 个全部可能位置；此时，核中心分别位于 `img[2,1]`、`img[2,2]`和 `img[2,3]`处。
- 图(d)是腐蚀结果 `rst`，即在 `kernel` 完全位于前景图像中时，将其中心点所对应的 `rst` 中像素点的值置为 1；当 `kernel` 不完全位于前景图像中时，将其中心点对应的 `rst` 中像素点的值置为 0。

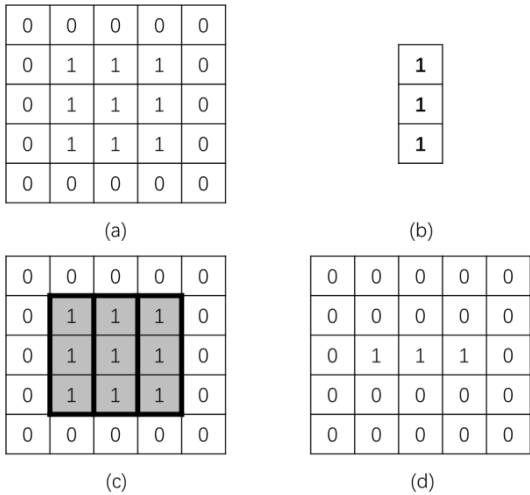


图 8-4 腐蚀示意图

在 OpenCV 中，使用函数 `cv2.erode()`实现腐蚀操作，其语法格式为：

```
dst = cv2.erode( src, kernel[, anchor[, iterations[, borderType[,  
borderValue]]]] )
```

式中：

- `dst` 是腐蚀后所输出的目标图像，该图像和原始图像具有同样的类型和大小。
- `src` 是需要进行腐蚀的原始图像，图像的通道数可以是任意的。但是要求图像的深度必须是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F`、`CV_64F` 中的一种。
- `kernel` 代表腐蚀操作时所采用的结构类型。它可以自定义生成，也可以通过函数 `cv2.getStructuringElement()`生成。

- anchor 代表 element 结构中锚点的位置。该值默认为(-1,-1)，在核的中心位置。
- iterations 是腐蚀操作迭代的次数，该值默认为 1，即只进行一次腐蚀操作。
- borderType 代表边界样式，一般采用其默认值 BORDER_CONSTANT。该项的具体值如表 8-1 所示。

表 8-1 borderType 值

类型	说明
cv2.BORDER_CONSTANT	iiii abcdefgh iiii, 特定值 i
cv2.BORDER_REPLICATE	aaaaa abcdefgh hhhhhhh
cv2.BORDER_REFLECT	fedcba abcdefgh hgfedcb
cv2.BORDER_WRAP	cdefgh abcdefgh abcdefg
cv2.BORDER_REFLECT_101	gfedcb abcdefgh gfedcba
cv2.BORDER_TRANSPARENT	uvwxyz abcdefgh ijklmno
cv2.BORDER_REFLECT101	与 BORDER_REFLECT_101 相同
cv2.BORDER_DEFAULT	与 BORDER_REFLECT_101 相同
cv2.BORDER_ISOLATED	不考虑 ROI (Region of Interest, 感兴趣区域) 之外的区域

- borderValue 是边界值，一般采用默认值。在 C++中提供了函数 morphologyDefaultBorderValue()来返回腐蚀和膨胀的“魔力 (magic)”边界值，Python 不支持该函数。

【例 8.1】使用数组演示腐蚀的基本原理。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img=np.zeros((5,5),np.uint8)
img[1:4,1:4]=1
kernel = np.ones((3,1),np.uint8)
erosion = cv2.erode(img,kernel)
print("img=\n",img)
print("kernel=\n",kernel)
print("erosion=\n",erosion)
```

运行程序，结果如下所示：

```
img=
[[0 0 0 0 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 0 0 0 0]]
kernel=
[[1]
 [1]
 [1]]
erosion=
[[0 0 0 0 0]
 [0 0 0 0 0]]
```



```
[0 1 1 1 0]
[0 0 0 0 0]
[0 0 0 0 0]]
```

从本例中可以看到，只有当核 kernel 的中心点位于 img 中的 img[2,1]、img[2,2]、img[2,3] 处时，核才完全处于前景图像中。所以，在腐蚀结果图像中，只有这三个点的值为 1，其余点的值皆为 0。

【例 8.2】使用函数 cv2.erode()完成图像腐蚀。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("erode.bmp",cv2.IMREAD_UNCHANGED)
kernel = np.ones((5,5),np.uint8)
erosion = cv2.erode(o,kernel)
cv2.imshow("original",o)
cv2.imshow("erosion",erosion)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，使用语句 kernel=np.ones((5,5),np.uint8)生成如图 8-5 所示的 5×5 的核，来对原始图像进行腐蚀。

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

图 8-5 自定义核

运行程序，结果如图 8-6 所示。其中，左图是原始图像，右图是腐蚀处理结果。从图中可以看到，腐蚀操作将原始图像内的毛刺腐蚀掉了。



图 8-6 腐蚀结果

【例 8.3】 调节函数 `cv2.erode()` 的参数，观察不同参数控制下的图像腐蚀效果。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("erode.bmp",cv2.IMREAD_UNCHANGED)
kernel = np.ones((9,9),np.uint8)
erosion = cv2.erode(o,kernel,iterations = 5)
cv2.imshow("original",o)
cv2.imshow("erosion",erosion)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，参数做了两个调整：

- 核的尺寸变为 9×9 。
- 使用参数 `iterations = 5` 对函数 `cv2.erode()` 的迭代次数进行控制，让其迭代 5 次。

运行程序，结果如图 8-7 所示。其中，左图是原始图像，右图是腐蚀处理结果。从图中可以看到，腐蚀操作将原始图像内的毛刺腐蚀掉了，而且，由于本例使用了更大的核、更多的迭代次数，所以图像被腐蚀得更严重了。



图 8-7 调节参数后的腐蚀结果

8.2 膨胀

膨胀操作是形态学中另外一种基本的操作。膨胀操作和腐蚀操作的作用是相反的，膨胀操作能对图像的边界进行扩张。膨胀操作将与当前对象（前景）接触到的背景点合并到当前对象内，从而实现将图像的边界点向外扩张。如果图像内两个对象的距离较近，那么在膨胀的过程中，两个对象可能会连通在一起。膨胀操作对填补图像分割后图像内所存在的空白相当有帮助。二值图像的膨胀示例如图 8-8 所示。

同腐蚀过程一样，在膨胀过程中，也是使用一个结构元来逐个像素地扫描要被膨胀的图像，并根据结构元和待膨胀图像的关系来确定膨胀结果。

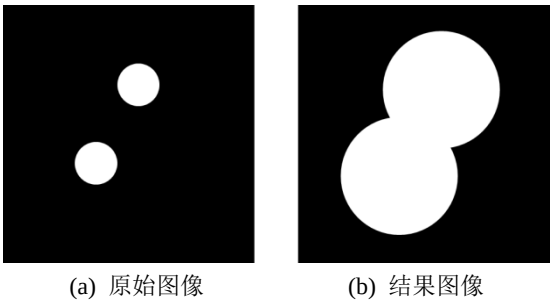


图 8-8 二值图像膨胀效果

例如，在图 8-9 中，整幅图像的背景色是黑色的，前景对象是一个白色的圆形。图像左上角的深色小块表示遍历图像所使用的结构元。在膨胀过程中，要将该结构元逐个像素地遍历整幅图像，并根据结构元与待膨胀图像的关系，来确定膨胀结果图像中与结构元中心点对应位置像素点的值。

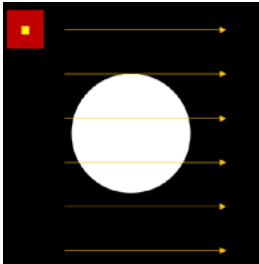


图 8-9 结构元与待膨胀图像

图 8-10 中的两幅图像代表结构元与前景色的两种不同关系。根据这两种不同关系来决定膨胀结果图像中，与结构元中心像素重合的点的像素值。

- 如果结构元中任意一点处于前景图像中，就将膨胀结果图像中对应像素点处理为前景色。
- 如果结构元完全处于背景图像外，就将膨胀结果图像中对应像素点处理为背景色。

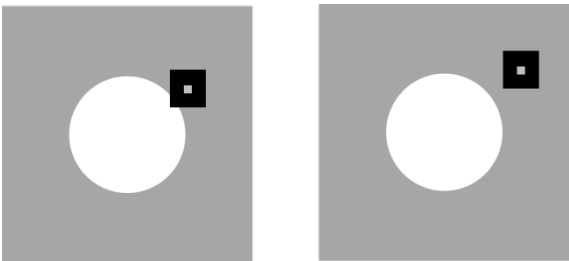


图 8-10 膨胀原理

针对图 8-10 中的图像，膨胀的结果就是前景对象的白色圆直径变大。上述结构元也被称为核。

例如，有待膨胀的图像 `img`，其值为：

```
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 1 1 1 0]]
```

```
[0 0 0 0 0]
[0 0 0 0 0]]
```

有一个结构元 `kernel`，其值为：

```
[[1]
[1]
[1]]
```

如果使用结构元 `kernel` 对图像 `img` 进行膨胀，则可以得到膨胀结果图像 `rst`：

```
[[0 0 0 0 0]
[0 1 1 1 0]
[0 1 1 1 0]
[0 1 1 1 0]
[0 0 0 0 0]]
```

这是因为当结构元 `kernel` 在图像 `img` 内逐个像素地进行遍历时，当核 `kernel` 的中心点 `kernel[1,0]` 位于 `img` 中的 `img[1,1]`、`img[1,2]`、`img[1,3]`、`img[2,1]`、`img[2,2]`、`img[2,3]`、`img[3,1]`、`img[3,2]` 或 `img[3,3]` 处时，核内像素点都存在与前景对象重合的像素点。所以，在膨胀结果图像中，这 9 个像素点的值被处理为 1，其余像素点的值被处理为 0。

上述示例的示意图如图 8-11 所示，其中：

- 图(a)表示待膨胀的 `img`。
- 图(b)是核 `kernel`。
- 图(c)中的阴影部分是 `kernel` 在遍历 `img` 时，`kernel` 中心像素点位于 `img[1,1]`、`img[3,3]` 时与前景色存在重合像素点的两种可能情况，实际上共有 9 个这样的与前景对象重合的可能位置。核 `kernel` 的中心分别位于 `img[1,1]`、`img[1,2]`、`img[1,3]`、`img[2,1]`、`img[2,2]`、`img[2,3]`、`img[3,1]`、`img[3,2]` 或 `img[3,3]` 时，核内像素点都存在与前景图像重合的像素点。
- 图(d)是膨胀结果图像 `rst`。在 `kernel` 内，当任意一个像素点与前景对象重合时，其中心点所对应的膨胀结果图像内的像素点值的为 1；当 `kernel` 与前景对象完全无重合时，其中心点对应的膨胀结果图像内像素点的值为 0。

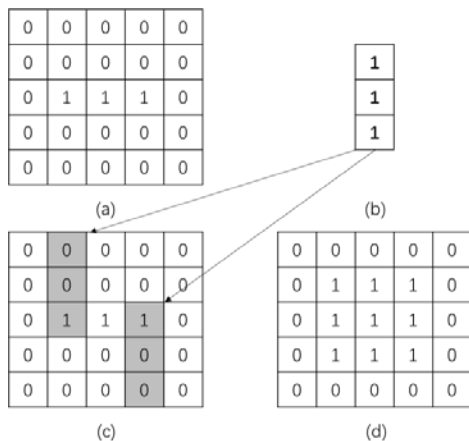


图 8-11 膨胀示意图



在 OpenCV 内，采用函数 `cv2.dilate()` 实现对图像的膨胀操作，该函数的语法结构为：

```
dst = cv2.dilate( src, kernel[, anchor[, iterations[, borderType[,
borderValue]]]])
```

式中：

- `dst` 代表膨胀后所输出的目标图像，该图像和原始图像具有同样的类型和大小。
- `src` 代表需要进行膨胀操作的原始图像。图像的通道数可以是任意的，但是要求图像的深度必须是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F`、`CV_64F` 中的一种。
- `element` 代表膨胀操作所采用的结构类型。它可以自定义生成，也可以通过函数 `cv2.getStructuringElement()` 生成。

参数 `kernel`、`anchor`、`iterations`、`borderType`、`borderValue` 与函数 `cv2.erode()` 内相应参数的含义一致。

【例 8.4】 使用数组演示膨胀的基本原理。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img=np.zeros((5,5),np.uint8)
img[2:3,1:4]=1
kernel = np.ones((3,1),np.uint8)
dilation = cv2.dilate(img,kernel)
print("img=\n",img)
print("kernel=\n",kernel)
print("dilation\n",dilation)
```

运行程序，结果如下所示：

```
img=
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 1 1 1 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
kernel=
[[1]
 [1]
 [1]]
dilation
[[0 0 0 0 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 1 1 1 0]
 [0 0 0 0 0]]
```

从本例中可以看到，只要当核 `kernel` 的任意一点处于前景图像中时，就将当前中心点所对应的膨胀结果图像内像素点的值置为 1。

【例 8.5】使用函数 `cv2.dilate()` 完成图像膨胀操作。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("dilation.bmp",cv2.IMREAD_UNCHANGED)
kernel = np.ones((9,9),np.uint8)
dilation = cv2.dilate(o,kernel)
cv2.imshow("original",o)
cv2.imshow("dilation",dilation)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，使用语句 `kernel=np.ones((9,9),np.uint8)` 生成 9×9 的核，来对原始图像进行膨胀操作。

运行程序，结果如图 8-12 所示。其中，左图是原始图像，右图是膨胀处理结果。从图中可以看到，膨胀操作将原始图像“变粗”了。



图 8-12 【例 8.5】对应的膨胀结果

【例 8.6】调节函数 `cv2.dilate()` 的参数，观察不同参数控制下的图像膨胀效果。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("dilation.bmp",cv2.IMREAD_UNCHANGED)
kernel = np.ones((5,5),np.uint8)
dilation = cv2.dilate(o,kernel,iterations = 9)
cv2.imshow("original",o)
cv2.imshow("dilation", dilation)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，参数做了两个调整：

- 核的大小变为 5×5 。
- 使用语句 `iterations = 9` 对迭代次数进行控制，让膨胀重复 9 次。

运行程序，结果如图 8-13 所示。其中，左图是原始图像，右图是膨胀处理结果。从图中可以看到，膨胀操作让原始图像实现了“生长”。在本例中，由于重复了 9 次，所以图像被膨



胀得更严重了。

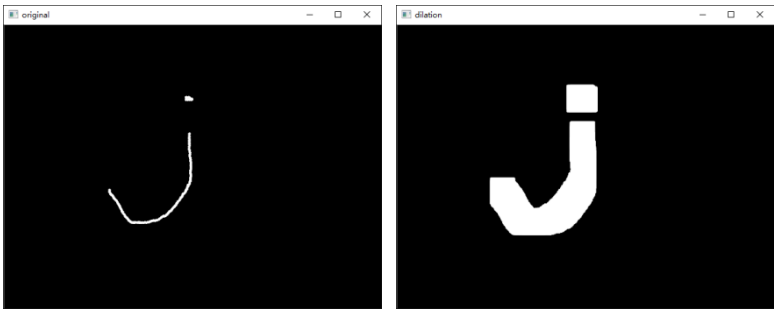


图 8-13 【例 8.6】对应的膨胀结果

8.3 通用形态学函数

腐蚀操作和膨胀操作是形态学运算的基础，将腐蚀和膨胀操作进行组合，就可以实现开运算、闭运算（关运算）、形态学梯度（Morphological Gradient）运算、礼帽运算（顶帽运算）、黑帽运算、击中击不中等多种不同形式的运算。

OpenCV 提供了函数 `cv2.morphologyEx()`来实现上述形态学运算，其语法结构如下：

```
dst = cv2.morphologyEx( src, op, kernel[, anchor[, iterations[, borderType[, borderValue]]]] )
```

式中：

- `dst` 代表经过形态学处理后所输出的目标图像，该图像和原始图像具有同样的类型和大小。
- `src` 代表需要进行形态学操作的原始图像。图像的通道数可以是任意的，但是要求图像的深度必须是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F`、`CV_64F` 中的一种。
- `op` 代表操作类型，如表 8-2 所示。各种形态学运算的操作规则均是将腐蚀和膨胀操作进行组合而得到的。

表 8-2 `op` 类型

类型	说明	含义	操作
<code>cv2.MORPH_ERODE</code>	腐蚀	腐蚀	<code>erode(src)</code>
<code>cv2.MORPH_DILATE</code>	膨胀	膨胀	<code>dilate(src)</code>
<code>cv2.MORPH_OPEN</code>	开运算	先腐蚀后膨胀	<code>dilate(erode(src))</code>
<code>cv2.MORPH_CLOSE</code>	闭运算	先膨胀后腐蚀	<code>erode(dilate(src))</code>
<code>cv2.MORPH_GRADIENT</code>	形态学梯度运算	膨胀图减腐蚀图	<code>dilate(src)-erode(src)</code>
<code>cv2.MORPH_TOPHAT</code>	顶帽运算	原始图像减开运算所得图像	<code>src-open(src)</code>
<code>cv2.MORPH_BLACKHAT</code>	黑帽运算	闭运算所得图像减原始图像	<code>close(src)-src</code>
<code>cv2.MORPH_HITMISS</code>	击中击不中	前景背景腐蚀运算的交集。仅仅支持 <code>CV_8UC1</code> 二进制图像	<code>intersection(erode(src),erode(srcI))</code>

- 参数 `kernel`、`anchor`、`iterations`、`borderType`、`borderValue` 与函数 `cv2.erode()`内相应参数的含义一致。

8.4 开运算

开运算进行的操作是先将图像腐蚀，再对腐蚀的结果进行膨胀。开运算可以用于去噪、计数等。

例如，在图 8-14 中，通过先腐蚀后膨胀的开运算操作实现了去噪，其中：

- 左图是原始图像。
- 中间的图是对原始图像进行腐蚀的结果。
- 右图是对腐蚀后的图像进行膨胀的结果，即对原始图像进行开运算的处理结果。



图 8-14 实现去噪的开运算

从图 8-14 中可以看到，原始图像在经过腐蚀、膨胀后实现了去噪的目的。除此以外，开运算还可以用于计数。例如，在对图 8-15 中的区域进行计数前，可以利用开运算将连接在一起的不同区域划分开，其中：

- 左图是原始图像。
- 中间的图是对原始图像进行腐蚀的结果。
- 右图是对腐蚀后的图像进行膨胀的结果，即对原始图像进行开运算的处理结果。



图 8-15 实现计数的开运算

通过将函数 `cv2.morphologyEx()` 中操作类型参数 `op` 设置为 “`cv2.MORPH_OPEN`”，可以实现开运算。其语法结构如下：

```
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
```

【例 8.7】 使用函数 `cv2.morphologyEx()` 实现开运算。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img1=cv2.imread("opening.bmp")
img2=cv2.imread("opening2.bmp")
```

```

k=np.ones((10,10),np.uint8)
r1=cv2.morphologyEx(img1,cv2.MORPH_OPEN,k)
r2=cv2.morphologyEx(img2,cv2.MORPH_OPEN,k)
cv2.imshow("img1",img1)
cv2.imshow("result1",r1)
cv2.imshow("img2",img2)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()

```

在本例中，分别针对两幅不同的图像做了开运算。运行程序，结果如图 8-16 所示，其中：

- 图(a)是原始图像 img1。
- 图(b)是原始图像 img1 经过开运算得到的图像 r1。
- 图(c)是原始图像 img2。
- 图(d)是原始图像 img2 经过开运算得到的图像 r2。

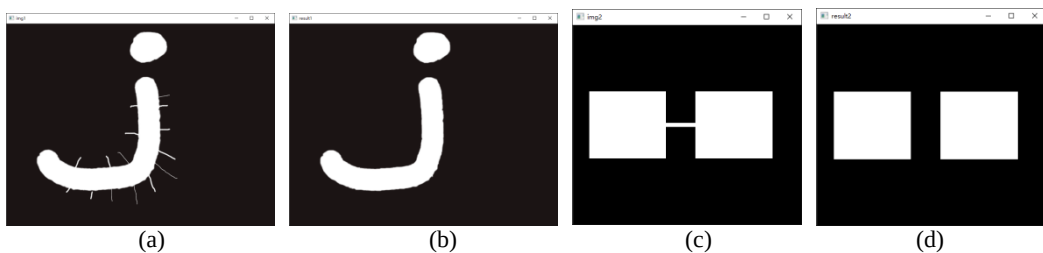


图 8-16 【例 8.7】对应的开运算结果

8.5 闭运算

闭运算是先膨胀、后腐蚀的运算，它有助于关闭前景物体内部的小孔，或去除物体上的小黑点，还可以将不同的前景图像进行连接。

例如，在图 8-17 中，通过先膨胀后腐蚀的闭运算去除了原始图像内部的小孔（内部闭合的闭运算），其中：

- 左图是原始图像。
- 中间的图是对原始图像进行膨胀的结果。
- 右图是对膨胀后的图像进行腐蚀的结果，即对原始图像进行闭运算的结果。

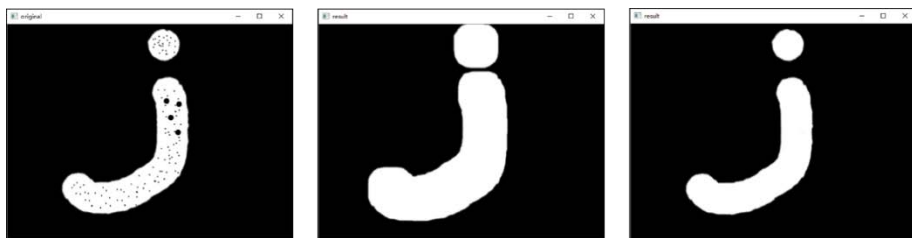


图 8-17 实现去除内部小孔的闭运算

从图 8-17 可以看到,原始图像在经过膨胀、腐蚀后,实现了闭合内部小孔的目的。除此以外,闭运算还可以实现前景图像的连接。例如,在图 8-18 中,利用闭运算将原本独立的两部分前景图像连接在一起,其中:

- 左图是原始图像。
- 中间的图是对原始图像进行膨胀的结果。
- 右图是对膨胀后的图像进行腐蚀的结果,即对原始图像进行闭运算的结果。



图 8-18 实现闭合连接的闭运算

通过将函数 `cv2.morphologyEx()` 中操作类型参数 `op` 设置为 “`cv2.MORPH_CLOSE`”, 可以实现闭运算。其语法结构如下:

```
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
```

【例 8.8】 使用函数 `cv2.morphologyEx()` 实现闭运算。

根据题目要求, 编写程序如下:

```
import cv2
import numpy as np
img1=cv2.imread("closing.bmp")
img2=cv2.imread("closing2.bmp")
k=np.ones((10,10),np.uint8)
r1=cv2.morphologyEx(img1,cv2.MORPH_CLOSE,k,iterations=3)
r2=cv2.morphologyEx(img2,cv2.MORPH_CLOSE,k,iterations=3)
cv2.imshow("img1",img1)
cv2.imshow("result1",r1)
cv2.imshow("img2",img2)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中, 分别针对两幅不同的图像做了闭运算。运行程序, 结果如图 8-19 所示, 其中:

- 图(a)是原始图像 `img1`。
- 图(b)是原始图像 `img1` 经过闭运算得到的图像 `r1`。
- 图(c)是原始图像 `img2`。
- 图(d)是原始图像 `img2` 经过闭运算得到的图像 `r2`。

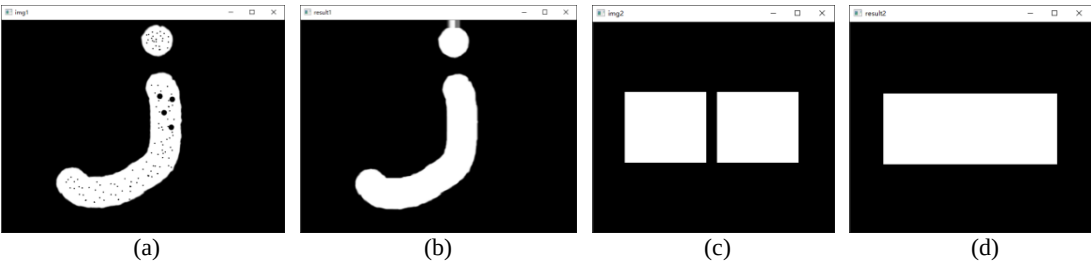


图 8-19 【例 8.8】对应的闭运算结果

8.6 形态学梯度运算

形态学梯度运算是用图像的膨胀图像减腐蚀图像的操作，该操作可以获取原始图像中前景图像的边缘。

例如，图 8-20 演示了形态学梯度运算。

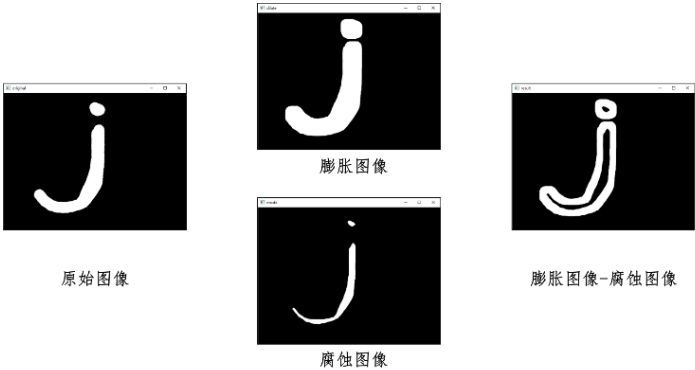


图 8-20 实现形态学梯度运算

从图 8-20 中可以看到，形态学梯度运算使用膨胀图像（扩张亮度）减腐蚀图像（收缩亮度），得到原始图像中前景对象的边缘。

通过将函数 `cv2.morphologyEx()` 的操作类型参数 `op` 设置为 “`cv2.MORPH_GRADIENT`”，可以实现形态学梯度运算。其语法结构如下：

```
result = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
```

【例 8.9】 使用函数 `cv2.morphologyEx()` 实现形态学梯度运算。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("gradient.bmp",cv2.IMREAD_UNCHANGED)
k=np.ones((5,5),np.uint8)
r=cv2.morphologyEx(o,cv2.MORPH_GRADIENT,k)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 8-21 所示，其中左图是原始图像，右图是形态学梯度运算结果。

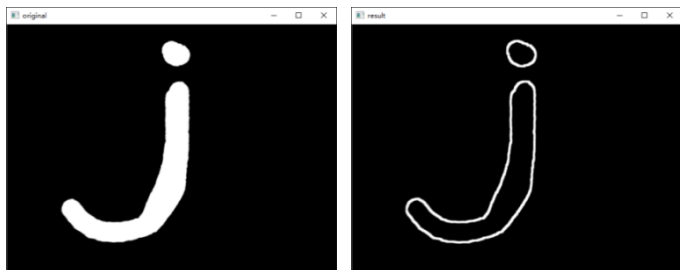


图 8-21 【例 8.9】程序的运行结果

8.7 礼帽运算

礼帽运算是用原始图像减去其开运算图像的操作。礼帽运算能够获取图像的噪声信息，或者得到比原始图像的边缘更亮的边缘信息。

例如，图 8-22 是一个礼帽运算示例，其中：

- 左图是原始图像。
- 中间的图是开运算图像。
- 右图是原始图像减开运算图像所得到的礼帽图像。



图 8-22 礼帽运算示意图

从图 8-22 中可以看到，礼帽运算使用原始图像减开运算图像得到礼帽图像，礼帽图像是原始图像中的噪声信息。

例如，在图 8-23 中，左图是原始图像，中间的图是开运算图像，右图是原始图像减开运算图像得到的礼帽图像，礼帽图像显示的是比原始图像的边缘更亮的边缘信息。

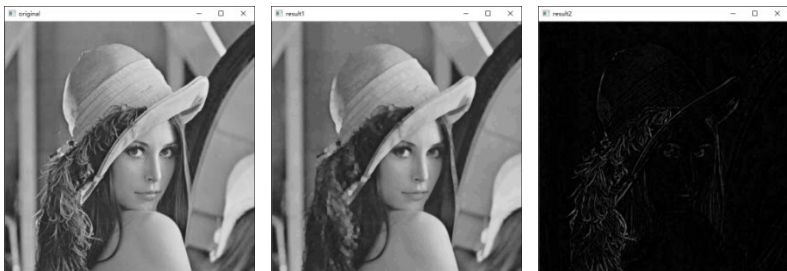


图 8-23 礼帽运算实例示意图

通过将函数 `cv2.morphologyEx()` 中操作类型参数 `op` 设置为“`cv2.MORPH_TOPHAT`”，可以

实现礼帽运算。其语法结构如下：

```
result = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
```

【例 8.10】 使用函数 `cv2.morphologyEx()` 实现礼帽运算。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o1=cv2.imread("tophat.bmp",cv2.IMREAD_UNCHANGED)
o2=cv2.imread("lena.bmp",cv2.IMREAD_UNCHANGED)
k=np.ones((5,5),np.uint8)
r1=cv2.morphologyEx(o1,cv2.MORPH_TOPHAT,k)
r2=cv2.morphologyEx(o2,cv2.MORPH_TOPHAT,k)
cv2.imshow("original1",o1)
cv2.imshow("original2",o2)
cv2.imshow("result1",r1)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 8-24 所示，其中：

- 图(a)是原始图像 o1。
- 图(b)是原始图像 o2。
- 图(c)是原始图像 o1 经过礼帽运算得到的图像 r1。
- 图(d)是原始图像 o2 经过礼帽运算得到的图像 r2。

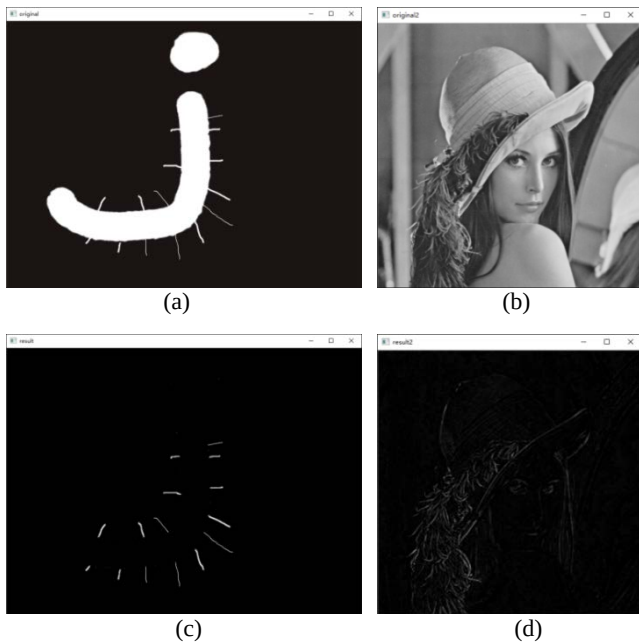


图 8-24 【例 8.10】程序的运行结果

8.8 黑帽运算

黑帽运算是用闭运算图像减去原始图像的操作。黑帽运算能够获取图像内部的小孔，或前景色中的小黑点，或者得到比原始图像的边缘更暗的边缘部分。

例如，图 8-25 所示是一个黑帽运算，其中：

- 左图是原始图像。
- 中间的图是闭运算图像。
- 右图是使用闭运算图像减原始图像所得到的黑帽图像。

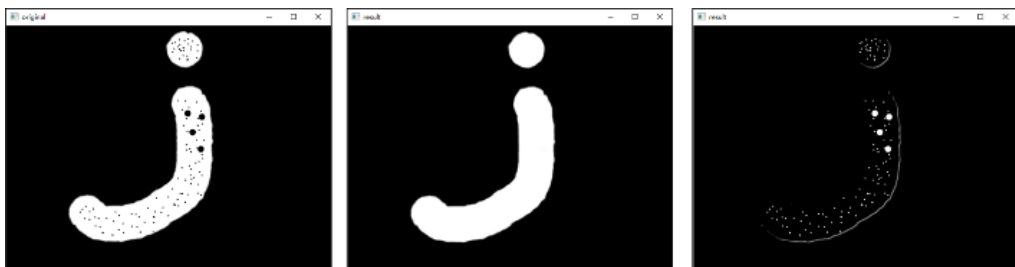


图 8-25 黑帽运算示意图

从图 8-25 可以看到，黑帽运算使用闭运算图像减原始图像得到黑帽图像，黑帽图像是原始图像中的小孔（噪声）。

例如，在图 8-26 中，左图是原始图像，中间的图是闭运算图像，右图是闭运算图像减原始图像得到的黑帽图像，黑帽图像显示的是比原始图像边缘更暗的边缘部分。



图 8-26 黑帽运算实例示意图

通过将函数 `cv2.morphologyEx()` 中操作类型参数 `op` 设置为 “`cv2.MORPH_BLACKHAT`”，可以实现黑帽运算。其语法结构如下：

```
result = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)
```

【例 8.11】 使用函数 `cv2.morphologyEx()` 实现黑帽运算。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o1=cv2.imread("blackhat.bmp",cv2.IMREAD_UNCHANGED)
o2=cv2.imread("lena.bmp",cv2.IMREAD_UNCHANGED)
```



```

k=np.ones((5,5),np.uint8)
r1=cv2.morphologyEx(o1,cv2.MORPH_BLACKHAT,k)
r2=cv2.morphologyEx(o2,cv2.MORPH_BLACKHAT,k)
cv2.imshow("original1",o1)
cv2.imshow("original2",o2)
cv2.imshow("result1",r1)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，结果如图 8-27 所示，其中：

- 图(a)是原始图像 o1。
- 图(b)是原始图像 o2。
- 图(c)是原始图像 o1 经过黑帽运算得到的图像 r1。
- 图(d)是原始图像 o2 经过黑帽运算得到的图像 r2。

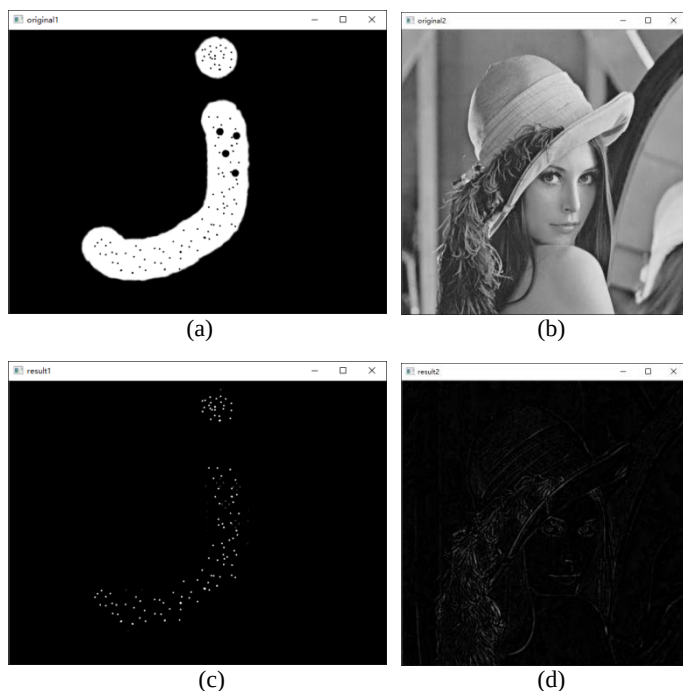


图 8-27 【例 8.11】程序的运行结果

8.9 核函数

在进行形态学操作时，必须使用一个特定的核（结构元）。该核可以自定义生成，也可以通过函数 `cv2.getStructuringElement()` 构造。函数 `cv2.getStructuringElement()` 能够构造并返回一个用于形态学处理所使用的结构元素。该函数的语法格式为：

```
retval = cv2.getStructuringElement( shape, ksize[, anchor])
```

该函数用来返回一个用于形态学操作的指定大小和形状的结构元素。

函数中的参数含义如下。

- `shape` 代表形状类型，其可能的取值如表 8-3 所示。

表 8-3 形状类型

类型	说明
<code>cv2.MORPH_RECT</code>	矩形结构元素。所有元素值都是 1
<code>cv2.MORPH_CROSS</code>	十字形结构元素。对角线元素值为 1
<code>cv2.MORPH_ELLIPSE</code>	椭圆形结构元素

- `ksize` 代表结构元素的大小。
- `anchor` 代表结构元素中的锚点位置。默认的值是`(-1, -1)`，是形状的中心。只有十字星型的形状与锚点位置紧密相关。在其他情况下，锚点位置仅用于形态学运算结果的调整。

当然，除了使用该函数，用户也可以自己构建任意二进制掩码作为形态学操作中所使用的结构元素。

【例 8.12】使用函数 `cv2.getStructuringElement()`生成不同结构的核。

根据题目要求，编写程序如下：

```
import cv2
kernel1 = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
kernel2 = cv2.getStructuringElement(cv2.MORPH_CROSS, (5,5))
kernel3 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
print("kernel1=\n",kernel1)
print("kernel2=\n",kernel2)
print("kernel3=\n",kernel3)
```

运行程序，结果如下：

```
kernel1=
[[1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]]
kernel2=
[[0 0 1 0 0]
 [0 0 1 0 0]
 [1 1 1 1 1]
 [0 0 1 0 0]
 [0 0 1 0 0]]
kernel3=
[[0 0 1 0 0]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 0 1 0 0]]
```



【例 8.13】 编写程序，观察不同的核对形态学操作的影响。

根据题目要求，编写程序如下：

```
import cv2
o=cv2.imread("kernel.bmp",cv2.IMREAD_UNCHANGED)
kernel1 = cv2.getStructuringElement(cv2.MORPH_RECT, (59,59))
kernel2 = cv2.getStructuringElement(cv2.MORPH_CROSS, (59,59))
kernel3 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (59,59))
dst1 = cv2.dilate(o,kernel1)
dst2 = cv2.dilate(o,kernel2)
dst3 = cv2.dilate(o,kernel3)
cv2.imshow("original",o)
cv2.imshow("dst1",dst1)
cv2.imshow("dst2",dst2)
cv2.imshow("dst3",dst3)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 8-28 所示，其中：

- 图(a)是原始图像 o1。
- 图(b)是使用矩形结构核对原始图像进行膨胀操作的结果 dst1。
- 图(c)是使用十字结构核对原始图像进行膨胀操作的结果 dst2。
- 图(d)是使用椭圆结构核对原始图像进行膨胀操作的结果 dst3。

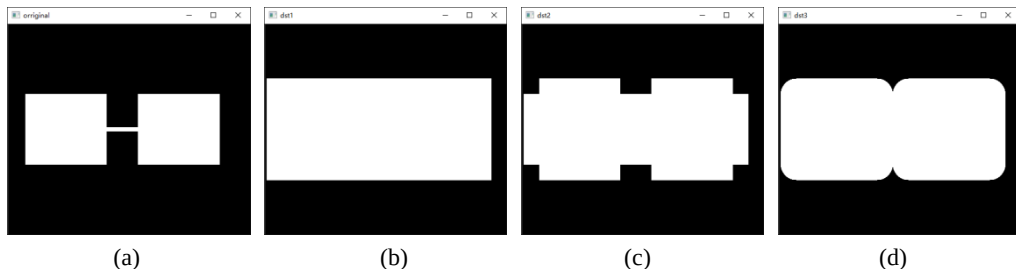


图 8-28 【例 8.13】程序的运行结果